

AWS CLOUD & DEVOPS TRAINING PROGRAM

Week 5 Training Report

Kubernetes Fundamentals & Application Deployment

Prepared by: Dilip

Program: AWS Cloud & DevOps Engineer Training

Institution: GIET Engineering College, Rajahmundry

Reporting Period: July 6, 2026 – July 11, 2026

GitHub: Dilip714

Table of Contents

1. Introduction & Learning Objectives

This report documents Week 5 of the AWS Cloud & DevOps Engineer training program, covering a complete, hands-on progression through Kubernetes — from cluster fundamentals to a full multi-tier application deployment using Helm.

1.1 Learning Objectives

The core focus of Week 5 was container orchestration with Kubernetes. The learning objectives for this module were:

- Understand Kubernetes architecture and core components.
- Deploy containerized applications on Kubernetes.
- Manage Pods, Deployments, ReplicaSets, and Services.
- Configure ConfigMaps and Secrets.
- Expose applications using Ingress.
- Monitor and troubleshoot Kubernetes workloads.

2. Day 1 – Kubernetes Fundamentals

2.1 Topics Covered

- What is Kubernetes?
- Kubernetes Architecture and Control Plane Components
- Worker Nodes, Pods, ReplicaSets, Deployments, and Namespaces

2.2 Implementation

A local single-node Kubernetes cluster was provisioned using Minikube with the Docker driver. After starting the cluster, the node status and cluster information were verified using kubectl, confirming that the control-plane node was in a Ready state and that the Kubernetes API server and CoreDNS were reachable.

```

Version:      0.19.0
GitCommit:    de40ad0
PS C:\WINDOWS\system32> minikube start --driver=docker
* minikube v1.38.1 on Microsoft Windows 11 Home Single Language 25H2
* Using the docker driver based on user configuration
* Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
* Using Docker Desktop driver with root privileges
* Starting "minikube" primary control-plane node in "minikube" cluster
* Pulling base image v0.0.50 ...
* Downloading Kubernetes v1.35.1 preload ...
  > preloaded-images-k8s-v18-v1...: 272.45 MiB / 272.45 MiB 100.00% 1.61 Mi
  > gcr.io/k8s-minikube/kicbase...: 519.58 MiB / 519.58 MiB 100.00% 2.18 Mi
* Creating docker container (CPUs=2, Memory=4000MB) ...
* Preparing Kubernetes v1.35.1 on Docker 29.2.1 ...
* Configuring bridge CNI (Container Networking Interface) ...
* Verifying Kubernetes components...
  - Using image gcr.io/k8s-minikube/storage-provisioner:v5
* Enabled addons: storage-provisioner, default-storageclass
* Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
PS C:\WINDOWS\system32> kubectl get nodes
NAME          STATUS    ROLES    AGE   VERSION
minikube      Ready     control-plane 33s   v1.35.1
PS C:\WINDOWS\system32> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:58770
CoreDNS is running at https://127.0.0.1:58770/api/v1/namespaces/kube-system/services/kube-dns:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\WINDOWS\system32> kubectl run nginx-pod --image=nginx
pod/nginx-pod created
PS C:\WINDOWS\system32> kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
nginx-pod     0/1     ContainerCreating 0           8s
PS C:\WINDOWS\system32> kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
nginx-pod     1/1     Running   0           84s
PS C:\WINDOWS\system32> kubectl describe pod nginx-pod
Name:         nginx-pod
Namespace:    default
Priority:      0
Service Account: default
Node:         minikube/192.168.49.2
Start Time:   Mon, 06 Jul 2026 22:39:01 +0530
Labels:       run=nginx-pod
Annotations:   <none>
Status:       Running
IP:           10.244.0.3
IPs:
  IP: 10.244.0.3
Containers:

```

Figure 1: Minikube cluster startup, node status, and initial Nginx Pod creation

A simple Nginx Pod was then deployed directly using the `kubectl run` command. The Pod transitioned from `ContainerCreating` to a `Running` state, and `kubectl describe pod` was used to inspect its metadata, node assignment, labels, and IP address in detail.

```
PS C:\WINDOWS\system32> kubectl get nodes
NAME        STATUS    ROLES    AGE   VERSION
minikube    Ready     control-plane  33s   v1.35.1
PS C:\WINDOWS\system32> kubectl cluster-info
Kubernetes control plane is running at https://127.0.0.1:58770
CoreDNS is running at https://127.0.0.1:58770/api/v1/namespaces/kube-system/services/kube-dns
:dns/proxy

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
PS C:\WINDOWS\system32> kubectl run nginx-pod --image=nginx
pod/nginx-pod created
PS C:\WINDOWS\system32> kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
nginx-pod   0/1     ContainerCreating  0          8s
PS C:\WINDOWS\system32> kubectl get pods
NAME        READY   STATUS    RESTARTS   AGE
nginx-pod   1/1     Running   0          84s
PS C:\WINDOWS\system32> kubectl describe pod nginx-pod
Name:         nginx-pod
Namespace:    default
Priority:      0
Service Account: default
Node:         minikube/192.168.49.2
Start Time:   Mon, 06 Jul 2026 22:39:01 +0530
Labels:       run=nginx-pod
Annotations:   <none>
Status:       Running
IP:           10.244.0.3
IPs:
  IP: 10.244.0.3
Containers:
  nginx-pod:
    Container ID:  docker://82b7e8daf4dfe6119b2e0b25e3d45bdc8c2e2ce3bbd5755efeb43ead755d57b5
    Image:         nginx
    Image ID:      docker-pullable://nginx@sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30
544d056b0371f8082f7bce
    Port:         <none>
    Host Port:     <none>
    State:         Running
      Started:     Mon, 06 Jul 2026 22:39:29 +0530
    Ready:         True
    Restart Count:  0
    Environment:   <none>
    Mounts:         <none>
```

Figure 2: Verifying Pod status and detailed Pod description output

The Pod's runtime logs were inspected using `kubectl logs`, confirming that the Nginx entrypoint scripts executed successfully and worker processes started as expected. An interactive shell session was then opened inside the container using `kubectl exec` to confirm the Nginx installation and explore the container's filesystem. Finally, `kubectl get pods -A` was used to list all Pods across every namespace, including core system components such as CoreDNS, etcd, the API server, controller manager, and kube-proxy.

```

PS C:\WINDOWS\system32> kubectl logs nginx-pod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configurat
ion
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.co
nf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/07/06 17:09:29 [notice] 1#1: using the "epoll" event method
2026/07/06 17:09:29 [notice] 1#1: nginx/1.31.2
2026/07/06 17:09:29 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/07/06 17:09:29 [notice] 1#1: OS: Linux 6.6.87.2-microsoft-standard-WSL2
2026/07/06 17:09:29 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/07/06 17:09:29 [notice] 1#1: start worker processes
2026/07/06 17:09:29 [notice] 1#1: start worker process 29
2026/07/06 17:09:29 [notice] 1#1: start worker process 30
2026/07/06 17:09:29 [notice] 1#1: start worker process 31
2026/07/06 17:09:29 [notice] 1#1: start worker process 32
2026/07/06 17:09:29 [notice] 1#1: start worker process 33
2026/07/06 17:09:29 [notice] 1#1: start worker process 34
2026/07/06 17:09:29 [notice] 1#1: start worker process 35
2026/07/06 17:09:29 [notice] 1#1: start worker process 36
PS C:\WINDOWS\system32> kubectl exec -it nginx-pod -- /bin/bash
root@nginx-pod:/# ls
bin  dev  docker-entrypoint.sh  home  lib64  mnt  proc  run  srv  tmp  var
boot  docker-entrypoint.d  etc  lib  media  opt  root  sbin  sys  usr
root@nginx-pod:/# pwd
/
root@nginx-pod:/# nginx -v
nginx version: nginx/1.31.2
root@nginx-pod:/# exit
exit
PS C:\WINDOWS\system32> kubectl get pods -A

```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
default	nginx-pod	1/1	Running	0	2m53s
kube-system	coredns-7d764666f9-lrd42	1/1	Running	0	3m39s
kube-system	etcd-minikube	1/1	Running	0	3m45s
kube-system	kube-apiserver-minikube	1/1	Running	0	3m44s
kube-system	kube-controller-manager-minikube	1/1	Running	0	3m46s
kube-system	kube-proxy-snnsd	1/1	Running	0	3m39s

Figure 3: Pod logs, interactive container shell session, and cluster-wide Pod listing

Access to the running Nginx Pod was verified directly through a web browser, confirming that the application was serving traffic correctly within the cluster.

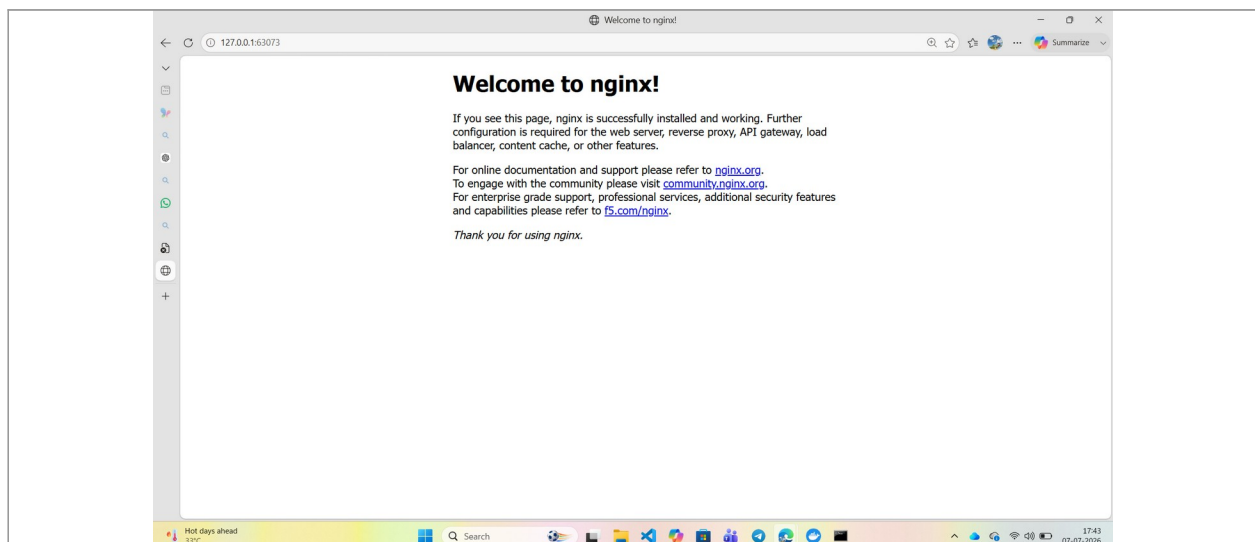


Figure 4: Browser verification of the running Nginx Pod

3. Day 2 – Deployments & Services

3.1 Topics Covered

- Deployments, ReplicaSets, and Rolling Updates
- Services: ClusterIP, NodePort, and LoadBalancer

3.2 Implementation

A Deployment named my-nginx was created and exposed as a NodePort Service to allow access from outside the cluster. The minikube service command was used to generate an accessible tunnel URL, confirming that traffic could reach the application through the exposed NodePort. The Deployment was then scaled from its initial replica count up to 5 replicas using `kubectl scale`, and `kubectl get pods` confirmed that all five Pod replicas reached a Running state, demonstrating high availability and horizontal scaling.

```

Administrator: Command Prompt
service/my-nginx exposed

C:\Windows\System32>kubectl get svc
NAME         TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)          AGE
kubernetes   ClusterIP   10.96.0.1    <none>        443/TCP          19h
my-nginx      NodePort    10.101.67.45 <none>        80:32140/TCP     7s

C:\Windows\System32>minikube service my-nginx

```

NAMESPACE	NAME	TARGET PORT	URL
default	my-nginx	80	http://192.168.49.2:32140

```

* Starting tunnel for service my-nginx.

```

NAMESPACE	NAME	TARGET PORT	URL
default	my-nginx		http://127.0.0.1:56326

```

* Opening service default/my-nginx in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
* Stopping tunnel for service my-nginx.

C:\Windows\System32>minikube service my-nginx

```

NAMESPACE	NAME	TARGET PORT	URL
default	my-nginx	80	http://192.168.49.2:32140

```

* Starting tunnel for service my-nginx.

```

NAMESPACE	NAME	TARGET PORT	URL
default	my-nginx		http://127.0.0.1:63073

```

* Opening service default/my-nginx in default browser...
! Because you are using a Docker driver on windows, the terminal needs to be open to run it.
* Stopping tunnel for service my-nginx.

C:\Windows\System32>kubectl scale deployment my-nginx --replicas=5
deployment.apps/my-nginx scaled

C:\Windows\System32>kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
my-nginx-575c69474d-4z5z9           1/1     Running   0          2m8s
my-nginx-575c69474d-62kx7           1/1     Running   0          6s
my-nginx-575c69474d-f8bbn           1/1     Running   0          6s
my-nginx-575c69474d-pjspr           1/1     Running   0          2m7s
my-nginx-575c69474d-s4zhd           1/1     Running   0          2m7s

```

Figure 5: NodePort Service exposure via Minikube tunnel and scaling the Deployment to 5 replicas

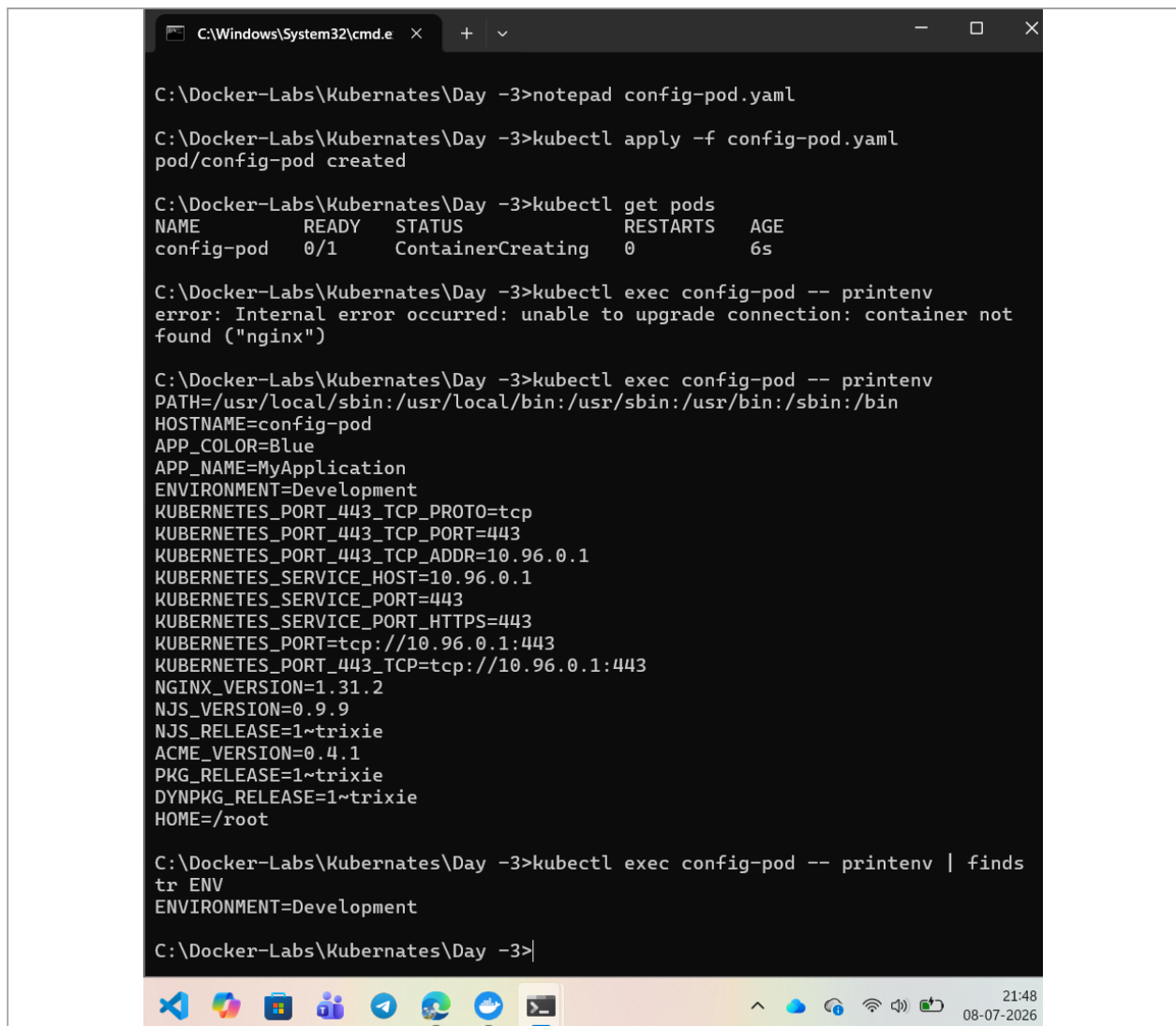
4. Day 3 – Configuration Management

4.1 Topics Covered

- ConfigMaps and Secrets
- Environment Variables
- Persistent Volumes (PV) and Persistent Volume Claims (PVC)

4.2 Implementation

A ConfigMap was created and injected into a Pod (config-pod) as environment variables. Using `kubectl exec` with `printenv`, the injected values — including custom application configuration such as `APP_NAME`, `APP_COLOR`, and `ENVIRONMENT` — were confirmed to be correctly available inside the container, alongside standard Kubernetes service-discovery variables.



```

C:\Windows\System32\cmd.e  x  +  v  -  □  x

C:\Docker-Labs\Kubernetes\Day -3>notepad config-pod.yaml

C:\Docker-Labs\Kubernetes\Day -3>kubectl apply -f config-pod.yaml
pod/config-pod created

C:\Docker-Labs\Kubernetes\Day -3>kubectl get pods
NAME          READY   STATUS             RESTARTS   AGE
config-pod    0/1     ContainerCreating   0           6s

C:\Docker-Labs\Kubernetes\Day -3>kubectl exec config-pod -- printenv
error: Internal error occurred: unable to upgrade connection: container not
found ("nginx")

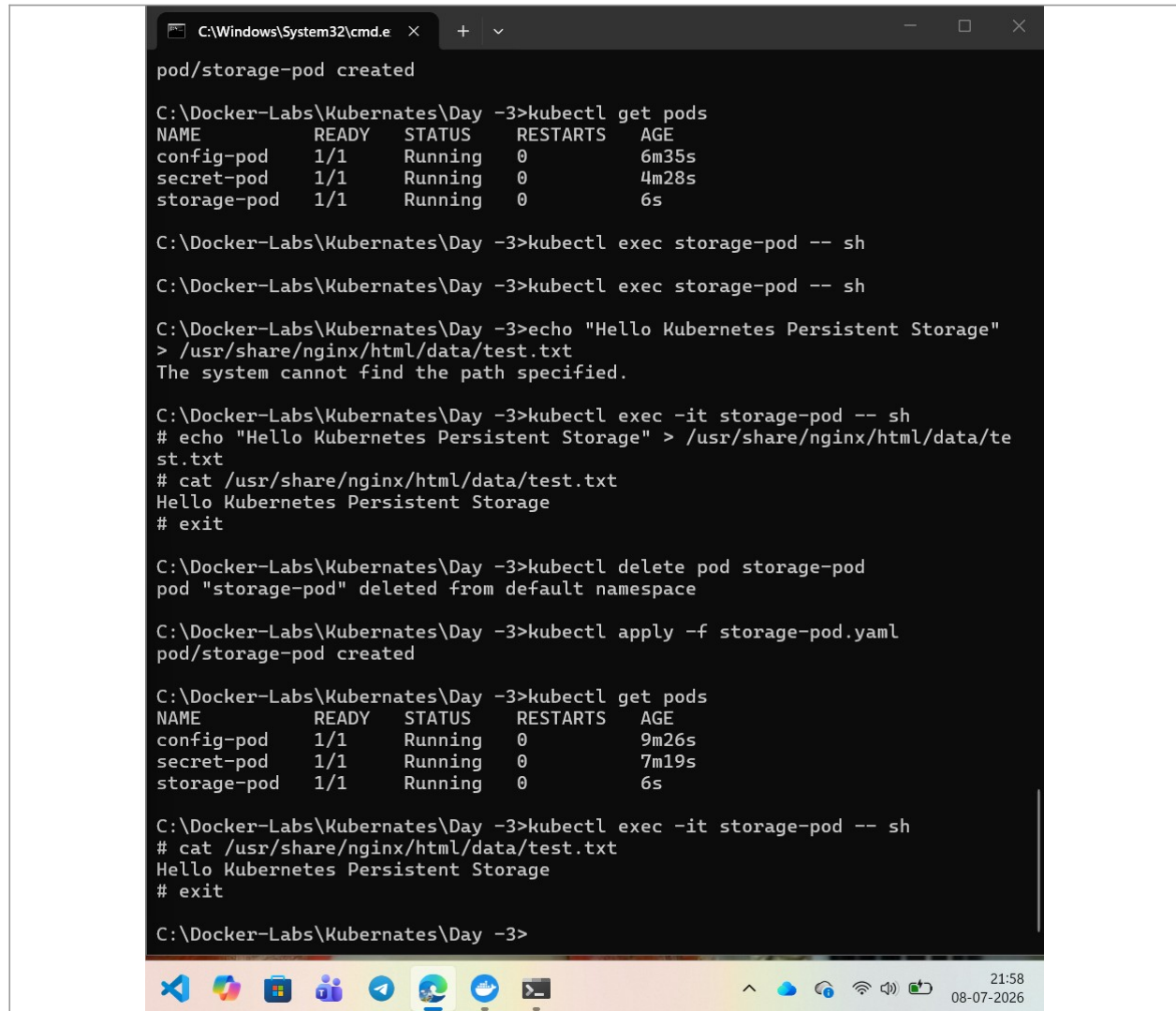
C:\Docker-Labs\Kubernetes\Day -3>kubectl exec config-pod -- printenv
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
HOSTNAME=config-pod
APP_COLOR=Blue
APP_NAME=MyApplication
ENVIRONMENT=Development
KUBERNETES_PORT_443_TCP_PROTO=tcp
KUBERNETES_PORT_443_TCP_PORT=443
KUBERNETES_PORT_443_TCP_ADDR=10.96.0.1
KUBERNETES_SERVICE_HOST=10.96.0.1
KUBERNETES_SERVICE_PORT=443
KUBERNETES_SERVICE_PORT_HTTPS=443
KUBERNETES_PORT=tcp://10.96.0.1:443
KUBERNETES_PORT_443_TCP=tcp://10.96.0.1:443
NGINX_VERSION=1.31.2
NJS_VERSION=0.9.9
NJS_RELEASE=1~trixie
ACME_VERSION=0.4.1
PKG_RELEASE=1~trixie
DYNPKG_RELEASE=1~trixie
HOME=/root

C:\Docker-Labs\Kubernetes\Day -3>kubectl exec config-pod -- printenv | find
tr ENV
ENVIRONMENT=Development

C:\Docker-Labs\Kubernetes\Day -3>
  
```

Figure 6: ConfigMap injection verified via environment variables inside the Pod

Persistent storage was then configured and validated. A storage-pod was created with a mounted Persistent Volume Claim, and a test file was written to the mounted path. After deleting and recreating the Pod from its manifest, the same file was confirmed to still be present, proving that data persisted independently of the Pod's lifecycle.



```
C:\Windows\System32\cmd.e  X  +  v

pod/storage-pod created

C:\Docker-Labs\Kubernetes\Day -3>kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
config-pod    1/1     Running   0           6m35s
secret-pod    1/1     Running   0           4m28s
storage-pod   1/1     Running   0           6s

C:\Docker-Labs\Kubernetes\Day -3>kubectl exec storage-pod -- sh

C:\Docker-Labs\Kubernetes\Day -3>kubectl exec storage-pod -- sh

C:\Docker-Labs\Kubernetes\Day -3>echo "Hello Kubernetes Persistent Storage"
> /usr/share/nginx/html/data/test.txt
The system cannot find the path specified.

C:\Docker-Labs\Kubernetes\Day -3>kubectl exec -it storage-pod -- sh
# echo "Hello Kubernetes Persistent Storage" > /usr/share/nginx/html/data/test.txt
# cat /usr/share/nginx/html/data/test.txt
Hello Kubernetes Persistent Storage
# exit

C:\Docker-Labs\Kubernetes\Day -3>kubectl delete pod storage-pod
pod "storage-pod" deleted from default namespace

C:\Docker-Labs\Kubernetes\Day -3>kubectl apply -f storage-pod.yaml
pod/storage-pod created

C:\Docker-Labs\Kubernetes\Day -3>kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
config-pod    1/1     Running   0           9m26s
secret-pod    1/1     Running   0           7m19s
storage-pod   1/1     Running   0           6s

C:\Docker-Labs\Kubernetes\Day -3>kubectl exec -it storage-pod -- sh
# cat /usr/share/nginx/html/data/test.txt
Hello Kubernetes Persistent Storage
# exit

C:\Docker-Labs\Kubernetes\Day -3>
```

Figure 7: Verifying data persistence across Pod recreation using a Persistent Volume Claim

5. Day 4 – Ingress & Networking

5.1 Topics Covered

- Ingress and Ingress Controllers
- DNS Basics
- Network Policies

5.2 Implementation

An Ingress Controller (NGINX) was installed, and path-based routing was configured so that requests to different paths were routed to different backend Services. Browser-based testing confirmed that a request to the /app2 path was correctly routed to the corresponding backend application.

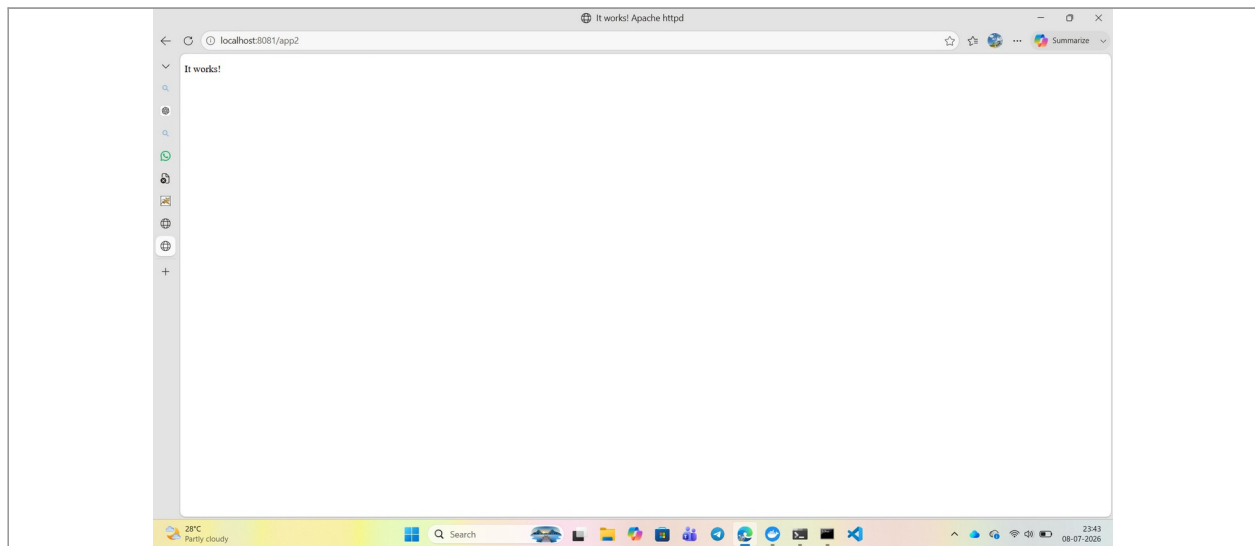


Figure 8: Browser verification of path-based routing to the app2 backend service

The underlying Services, Endpoints, and Ingress resources were inspected using kubectl. Two ClusterIP Services (app1 and app2) were confirmed to have healthy Endpoints, the NGINX Ingress Controller was verified as running with its NodePort Service, and the Ingress resource's rules were reviewed directly from the YAML manifest.

```

C:\Windows\System32\cmd.exe
Command Prompt - kubecti p
Command Prompt

Type      Reason      Age      From      Message
-----
Normal    Sync          6m19s   (x2 over 6m54s)   nginx-ingress-controller Scheduled for sync

C:\Docker-Labs\Kubernetes>kubectl get svc
NAME      TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE
app1      ClusterIP  10.98.208.117    <none>           80/TCP        12m
app2      ClusterIP  10.103.180.68    <none>           80/TCP        12m
kubernetes ClusterIP  10.96.0.1        <none>           443/TCP       97m

C:\Docker-Labs\Kubernetes>kubectl get endpoints
Warning: v1 Endpoints is deprecated in v1.33+; use discovery.k8s.io/v1 EndpointSlice
NAME      ENDPOINTS      AGE
app1      10.244.0.8:80   12m
app2      10.244.0.9:80   12m
kubernetes 192.168.49.2:8443 97m

C:\Docker-Labs\Kubernetes>kubectl get svc -n ingress-nginx
NAME                        TYPE      CLUSTER-IP      EXTERNAL-IP      PORT(S)      AGE
ingress-nginx-controller    NodePort   10.100.239.58    <none>           80:32563/TCP,443:38623/TCP 9m43s
ingress-nginx-controller-admission ClusterIP   10.107.131.50    <none>           443/TCP       9m43s

C:\Docker-Labs\Kubernetes>kubectl get ingress
NAME      CLASS  HOSTS      ADDRESS      PORTS      AGE
app-ingress  *      *          192.168.49.2  80         7m46s

C:\Docker-Labs\Kubernetes>type ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  ingressClassName: nginx
  rules:
  - http:
    paths:
    - path: /app1
      pathType: Prefix
      backend:

```

Figure 9: Services, Endpoints, Ingress Controller status, and Ingress manifest inspection

Additional path-specific Ingress rules were reviewed, followed by the creation of a Network Policy to restrict Pod-to-Pod communication. The deny-all policy was applied and described, confirming that it selected all Pods in the namespace and enforced ingress traffic isolation while leaving egress traffic unaffected.

```

C:\Windows\System32\cmd.exe
Command Prompt - kubecti p
Command Prompt

C:\Docker-Labs\Kubernetes>type app1-ingress.yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app1-ingress
spec:
  ingressClassName: nginx
  rules:
  - http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: app1
          port:
            number: 80

C:\Docker-Labs\Kubernetes>notepad network-policy.yaml
C:\Docker-Labs\Kubernetes>kubectl apply -f network-policy.yaml
networkpolicy.networking.k8s.io/deny-all created

C:\Docker-Labs\Kubernetes>kubectl get networkpolicy
NAME      POD-SELECTOR  AGE
deny-all <none>        6s

C:\Docker-Labs\Kubernetes>kubectl describe networkpolicy deny-all
Name:      deny-all
Namespace: default
Created on: 2026-07-08 23:48:17 +0530 IST
Labels:    <none>
Annotations: <none>
Spec:
  PodSelector: <none> (Allowing the specific traffic to all pods in this namespace)
  Ingress:
    - Allow ingress traffic:
      - <none> (Selected pods are isolated for ingress connectivity)
    - Not affecting egress traffic
  Policy Types: Ingress

```

Figure 10: Ingress manifest review and Network Policy configuration for restricting Pod communication

6. Day 5 – Monitoring & Troubleshooting

6.1 Topics Covered

- kubectl Debugging Commands
- Resource Requests & Limits
- Liveness and Readiness Probes
- Kubernetes Events and Logs

6.2 Implementation

A Readiness Probe was configured on a test Pod using an HTTP GET check, with defined delay, timeout, and success/failure thresholds. kubectl describe pod confirmed the probe configuration and that the Pod correctly transitioned to a Ready state once the health check succeeded.

```

C:\Windows\System32\cmd.exe
C:\Docker-Labs\Kubernetes\Day-5>notepad readiness.yaml
C:\Docker-Labs\Kubernetes\Day-5>kubectl apply -f readiness.yaml
pod/readiness-pod created
C:\Docker-Labs\Kubernetes\Day-5>kubectl get pods
NAME          READY   STATUS    RESTARTS   AGE
app1-867c-fbd96f-6pjj7  1/1     Running   1 (5ms ago)  23h
app2-649884f9d4-gtd2n  1/1     Running   1 (5s ago)   23h
liveness-pod  1/1     Running   0           61s
readiness-pod  9/1     Running   0           8s
resource-pod   1/1     Running   0           3m52s
C:\Docker-Labs\Kubernetes\Day-5>kubectl describe pod readiness-pod
Name:          readiness-pod
Namespace:     default
Priority:       0
Service Account: default
Node:          minikube/192.168.49.2
Start Time:    Thu, 09 Jul 2026 22:21:54 +0530
Labels:        <none>
Annotations:   <none>
Status:        Running
IP:            10.244.0.19
IPs:           10.244.0.19
Containers:
  nginx:
    Container ID:  docker://387913fa2a23577096093762c32bb680b2d2abb4abcee9b28906ab322cec254c
    Image:          nginx
    Image ID:       docker-pullable://nginx@sha256:ec4ed8b5299e5e90694af7750eb6dffd2627317d30544d056b0371f8082f7bce
    Port:           <none>
    Host Port:      <none>
    State:          Running
      Started:      Thu, 09 Jul 2026 22:21:57 +0530
    Ready:          True
    Restart Count:  0
    Readiness:      http-get http://:80/ delay=5s timeout=1s period=5s #success=1 #failure=3
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-qp4mb (ro)
  
```

Figure 11: Readiness Probe configuration and Pod status verification

Cluster resource monitoring was validated using the Metrics API and kubectl top pod, which displayed live CPU and memory consumption for each running Pod. A deliberately misconfigured Pod (bad-image) was also created to practice troubleshooting failure states, showing an ImagePullBackOff status that would typically be resolved by inspecting kubectl describe pod events and correcting the image reference.

```

C:\Docker-Labs\Kubernetes\Day-5>kubectl get apiservices
NAME                SERVICE      AVAILABLE   AGE
v1.                  Local        True        24h
v1.admissionregistration.k8s.io  Local        True        24h
v1.apixtensions.k8s.io  Local        True        24h
v1.apps              Local        True        24h
v1.authentication.k8s.io  Local        True        24h
v1.authorization.k8s.io  Local        True        24h
v1.autoscaling        Local        True        24h
v1.batch              Local        True        24h
v1.certificates.k8s.io  Local        True        24h
v1.coordination.k8s.io  Local        True        24h
v1.discovery.k8s.io    Local        True        24h
v1.events.k8s.io       Local        True        24h
v1.flowcontrol.apiserver.k8s.io  Local        True        24h
v1.networking.k8s.io   Local        True        24h
v1.node.k8s.io         Local        True        24h
v1.policy              Local        True        24h
v1.rbac.authorization.k8s.io  Local        True        24h
v1.resource.k8s.io     Local        True        24h
v1.scheduling.k8s.io   Local        True        24h
v1.storage.k8s.io      Local        True        24h
v1beta1.metrics.k8s.io  kube-system/metrics-server  True        104s
v2.autoscaling         Local        True        24h

C:\Docker-Labs\Kubernetes\Day-5>kubectl top pod
NAME                CPU(cores)   MEMORY(bytes)
app1-867c-fb96f-6p9g7  0m           20Mi
app2-649884f9d4-gtd2n  1m           9Mi
liveness-pod         1m           7Mi
readiness-pod         1m           7Mi
resource-pod          0m           7Mi

C:\Docker-Labs\Kubernetes\Day-5>kubectl get pods
NAME                READY   STATUS    RESTARTS   AGE
app1-867c-fb96f-6p9g7  1/1     Running   1 (13m ago)  23h
app2-649884f9d4-gtd2n  1/1     Running   1 (13m ago)  23h
bad-image              0/1     ImagePullBackOff  0           7m31s
liveness-pod           1/1     Running   0           9m49s
readiness-pod          1/1     Running   0           8m54s

```

Figure 12: Metrics API availability, live resource usage via `kubectl top`, and troubleshooting a failing Pod

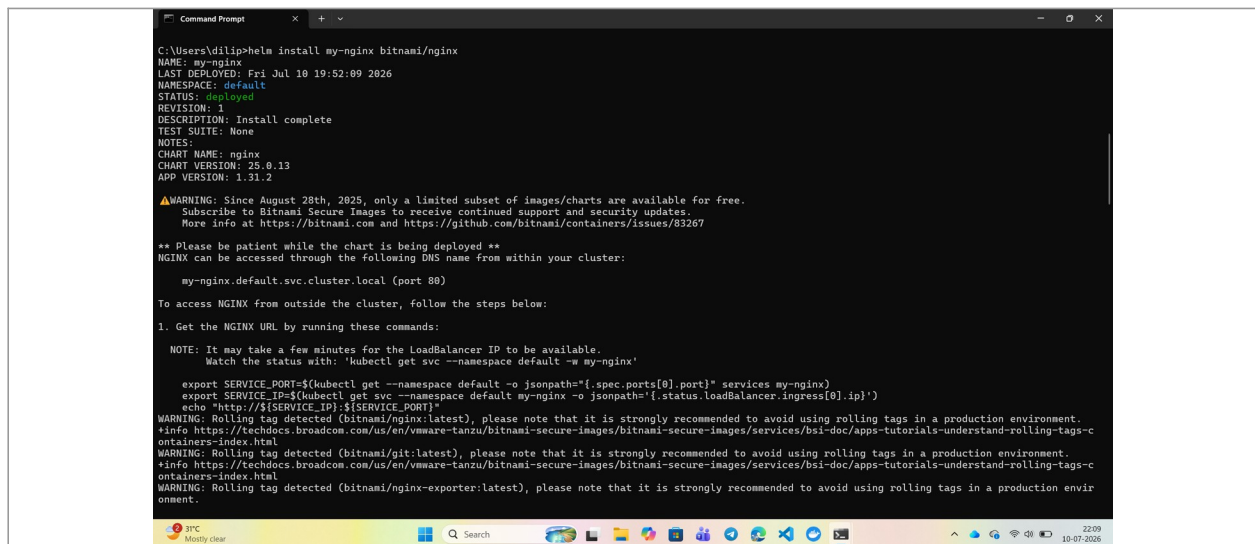
7. Day 6 – Helm Package Manager

7.1 Topics Covered

- Helm, Charts, Templates, and Values.yaml
- Releases, Upgrades, and Rollbacks

7.2 Implementation

Helm was used to install a pre-built Nginx chart from the Bitnami repository. The installation output confirmed a successful deployment, including the release name, chart version, application version, and instructions for accessing the service both within and outside the cluster.



```

C:\Users\dilip>helm install my-nginx bitnami/nginx
NAME: my-nginx
LAST DEPLOYED: Fri Jul 10 19:52:09 2026
NAMESPACE: default
STATUS: deployed
REVISION: 1
DESCRIPTION: Install complete
TEST SUITE: None
NOTES:
CHART NAME: nginx
CHART VERSION: 25.0.13
APP VERSION: 1.31.2

⚠️WARNING: Since August 28th, 2025, only a limited subset of images/charts are available for free.
  Subscribe to Bitnami Secure Images to receive continued support and security updates.
  More info at https://bitnami.com and https://github.com/bitnami/containers/issues/83267

** Please be patient while the chart is being deployed **
NGINX can be accessed through the following DNS name from within your cluster:

    my-nginx.default.svc.cluster.local (port 80)

To access NGINX from outside the cluster, follow the steps below:

1. Get the NGINX URL by running these commands:

    NOTE: It may take a few minutes for the LoadBalancer IP to be available.
    Watch the status with: 'kubectl get svc --namespace default -w my-nginx'

    export SERVICE_PORT=$(kubectl get --namespace default -o jsonpath="{.spec.ports[0].port}" services my-nginx)
    export SERVICE_IP=$(kubectl get svc --namespace default my-nginx -o jsonpath="{.status.loadBalancer.ingress[0].ip}")
    echo "http://$SERVICE_IP:$SERVICE_PORT"
⚠️WARNING: Rolling tag detected (bitnami/nginx:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-c
ontainers-index.html
⚠️WARNING: Rolling tag detected (bitnami/git:latest), please note that it is strongly recommended to avoid using rolling tags in a production environment.
+info https://techdocs.broadcom.com/us/en/vmware-tanzu/bitnami-secure-images/bitnami-secure-images/services/bsi-doc/apps-tutorials-understand-rolling-tags-c
ontainers-index.html
⚠️WARNING: Rolling tag detected (bitnami/nginx-exporter:latest), please note that it is strongly recommended to avoid using rolling tags in a production env
ironment.
  
```

Figure 13: Helm installation of the Bitnami Nginx chart

A custom Helm chart (mychart) was then created and explored in Visual Studio Code. The Deployment template made extensive use of Helm's templating engine — including include, with, and toYaml functions — to dynamically generate the replica count, container image, service account, security context, and labels from values supplied in values.yaml.

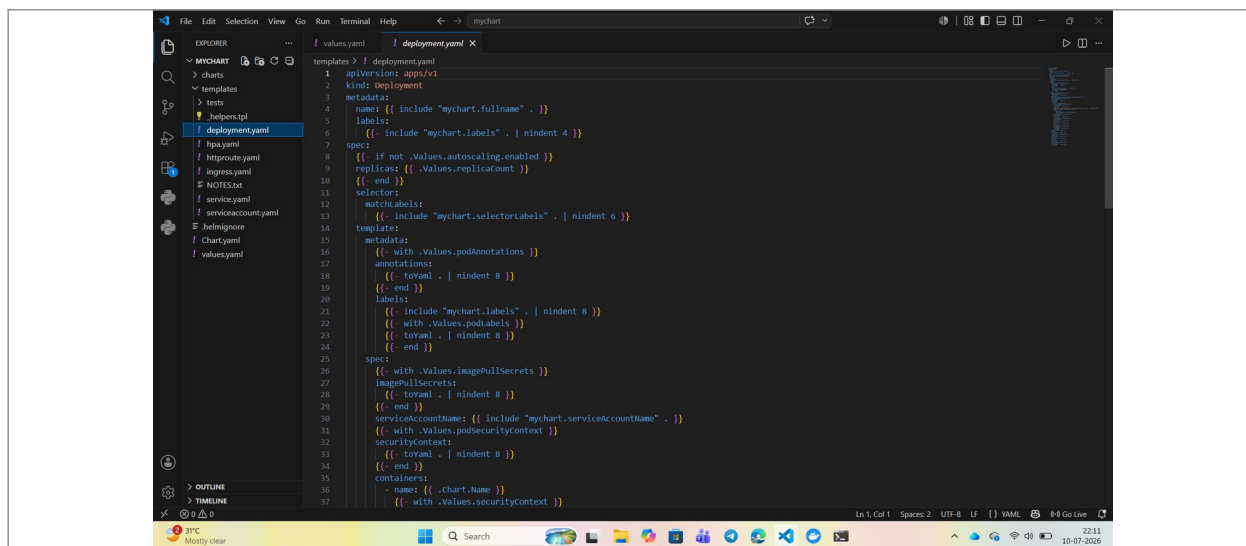


Figure 14: Custom Helm chart deployment.yaml template using Helm templating functions

The corresponding values.yaml file was reviewed, defining configurable parameters such as replica count, image repository and tag, image pull secrets, service account settings, and pod-level annotations and labels — all designed to be overridden at install or upgrade time without modifying the templates themselves.

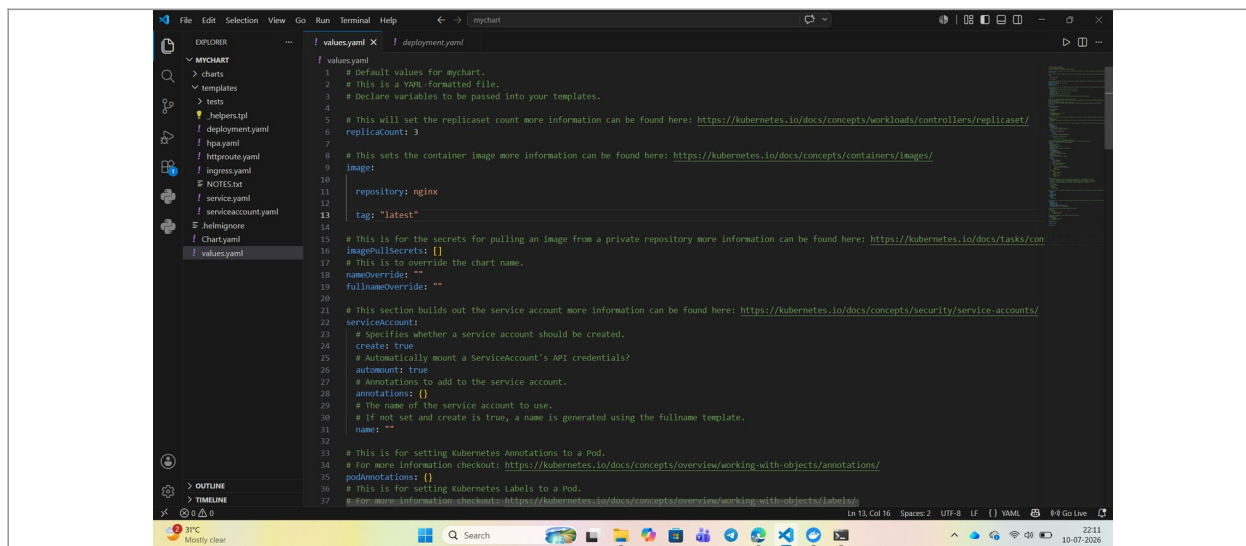


Figure 15: values.yaml defining configurable chart parameters

Helm's release management capabilities were then exercised on a release named myapp: the release was upgraded, rolled back to a previous revision using helm rollback, and the full revision history was reviewed with helm history, confirming each install, upgrade, and rollback event along with its status and description.

```

C:\Windows\System32\cmd.exe x + ~
4 Fri Jul 10 22:05:48 2026 deployed mychart-0.1.0 1.16.0 Upgrade complete

C:\Docker-Labs\Kubernetes\Day-6\mychart>
C:\Docker-Labs\Kubernetes\Day-6\mychart># 5. Rollback
'#' is not recognized as an internal or external command,
operable program or batch file.

C:\Docker-Labs\Kubernetes\Day-6\mychart>helm rollback myapp 1
Rollback was a success! Happy Helming!

C:\Docker-Labs\Kubernetes\Day-6\mychart>
C:\Docker-Labs\Kubernetes\Day-6\mychart># 6. Verify again
'#' is not recognized as an internal or external command,
operable program or batch file.

C:\Docker-Labs\Kubernetes\Day-6\mychart>kubectl get pods
NAME READY STATUS RESTARTS AGE
apli-867-fbd96f-6pjq7 1/1 Running 2 (23h ago) 47h
app2-649884f9du-gtd2n 1/1 Running 2 (23h ago) 47h
bad-image 0/1 ImagePullBackOff 0 23h
liveness-pod 1/1 Running 1 (23h ago) 23h
my-nginx-5dd58ffdf5f-5lkfk 1/1 Running 0 133m
myapp-mychart-5cc69755d8-hmxj7 1/1 Running 0 61s
myapp-mychart-5cc69755d8-lhwrz 1/1 Running 0 60s
myapp-mychart-5cc69755d8-phh6r 0/1 Terminating 0 0s
myapp-mychart-fbdf57c7d-49vqg 0/1 Terminating 0 0s
readiness-pod 1/1 Running 1 (23h ago) 23h
resource-pod 1/1 Running 1 (23h ago) 23h

C:\Docker-Labs\Kubernetes\Day-6\mychart>
C:\Docker-Labs\Kubernetes\Day-6\mychart># 7. Check history
'#' is not recognized as an internal or external command,
operable program or batch file.

C:\Docker-Labs\Kubernetes\Day-6\mychart>helm history myapp
REVISION UPDATED STATUS CHART APP VERSION DESCRIPTION
1 Fri Jul 10 22:01:59 2026 superseded mychart-0.1.0 1.16.0 Install complete
2 Fri Jul 10 22:04:20 2026 superseded mychart-0.1.0 1.16.0 Upgrade complete
3 Fri Jul 10 22:04:47 2026 superseded mychart-0.1.0 1.16.0 Rollback to 1
4 Fri Jul 10 22:05:48 2026 superseded mychart-0.1.0 1.16.0 Upgrade complete
5 Fri Jul 10 22:05:49 2026 deployed mychart-0.1.0 1.16.0 Rollback to 1

```

Figure 16: Helm rollback operation and full release revision history

8. Day 7 – Final Kubernetes Project

8.1 Project Objective

The capstone project for the Kubernetes module involved designing and deploying a complete multi-tier application — comprising a Frontend, a Backend API, and a MySQL database — using Kubernetes manifests and a custom Helm chart. The deliverables included Deployments, Services, ConfigMaps, Secrets, Persistent Volumes, Ingress, a packaged Helm Chart, an architecture diagram, a deployment guide, a README, and supporting screenshots, all published to a GitHub repository.

8.2 Implementation

The Helm chart for the final project was scaffolded and its values.yaml configured to define persistent storage (size and access mode) and Ingress settings (class name, path, and path type) for the application. An initial helm lint check surfaced a templating issue in the generated service.yaml that was subsequently resolved.

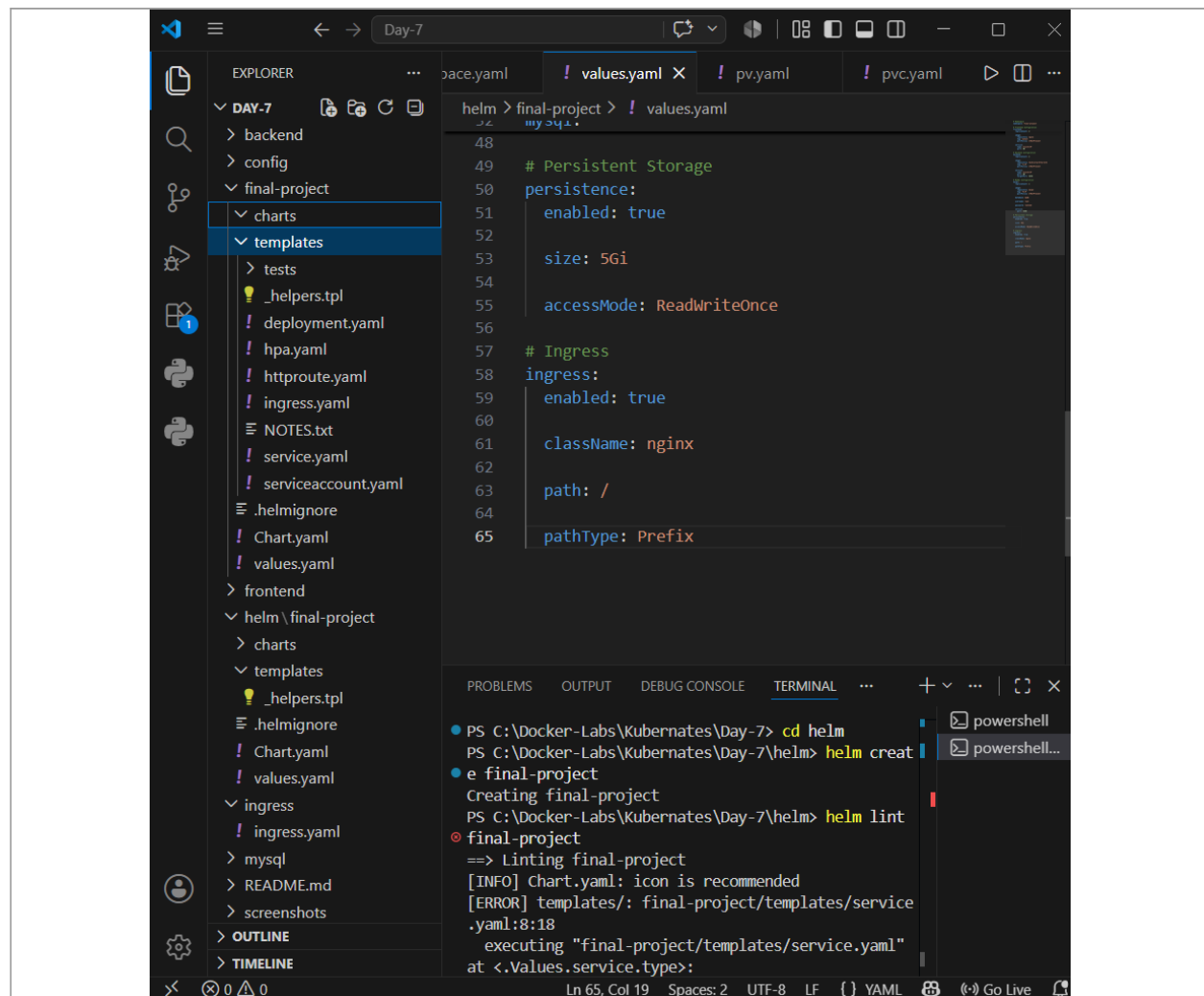


Figure 17: Final project Helm chart structure and values.yaml configuration in VS Code

The MySQL component was deployed using a dedicated Deployment manifest, with credentials and database name injected via Kubernetes Secrets and persistent storage mounted at the MySQL data directory through a Persistent Volume Claim. Once the MySQL Deployment reported as available, its corresponding ClusterIP Service was created and verified.

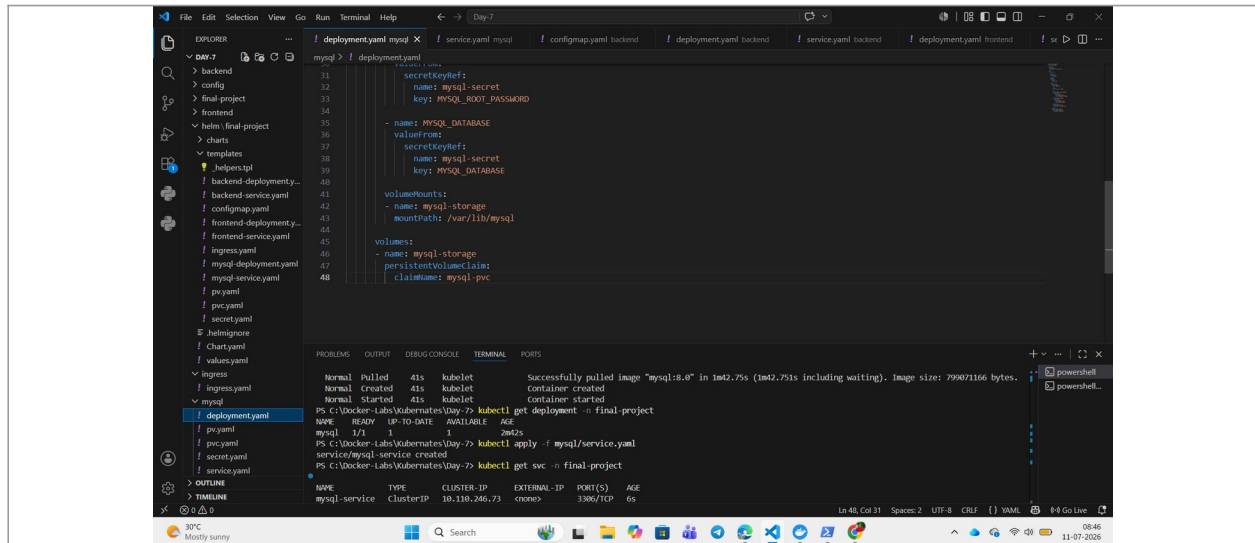


Figure 18: MySQL Deployment manifest, successful rollout, and Service creation

The Frontend Deployment manifest was configured with its container image, pull policy, and exposed port, completing the three-tier architecture. End-to-end connectivity was validated by browsing to the Frontend service and issuing a curl request to the Backend service, which returned the expected response, confirming that all three tiers — Frontend, Backend, and MySQL — were correctly deployed and able to communicate within the cluster.

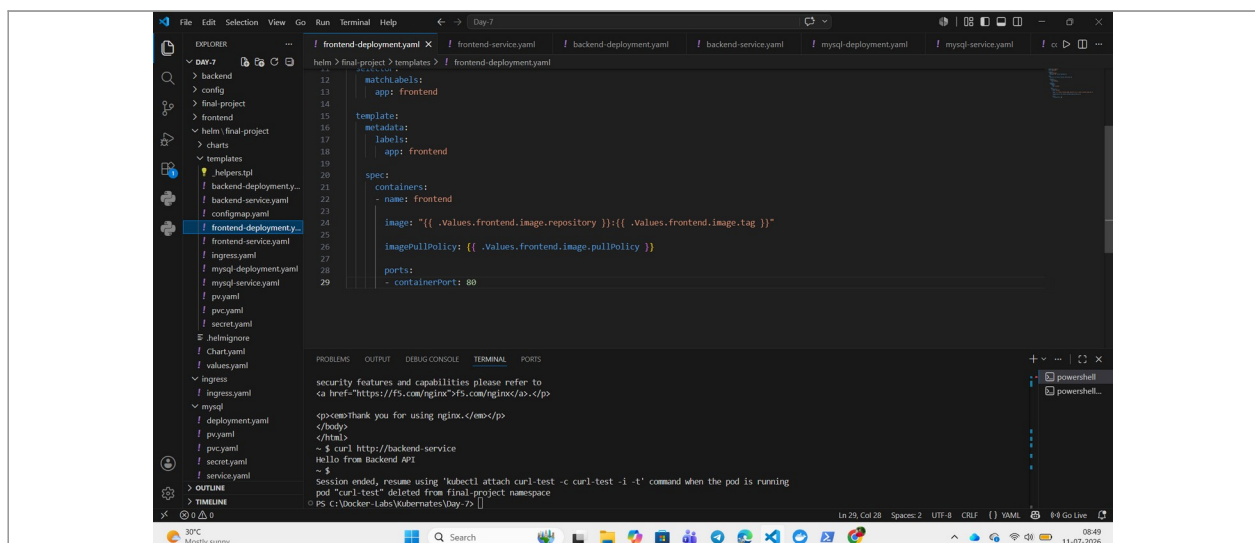


Figure 19: Frontend Deployment manifest and end-to-end connectivity verification with the Backend service

8.3 System Architecture Diagram

The complete system architecture for the final multi-tier project is illustrated below. External traffic enters through the NGINX Ingress Controller, which performs path-based routing to the Frontend and Backend Services. The Frontend communicates with the Backend API over its ClusterIP Service, and the Backend in turn queries MySQL through its own ClusterIP Service. The MySQL Pod persists data to a Persistent Volume via a Persistent Volume Claim, ensuring durability across Pod restarts. Application configuration and database credentials are externalized through a ConfigMap and a Secret respectively, and injected into the relevant Pods as environment variables rather than hard-coded into the manifests.

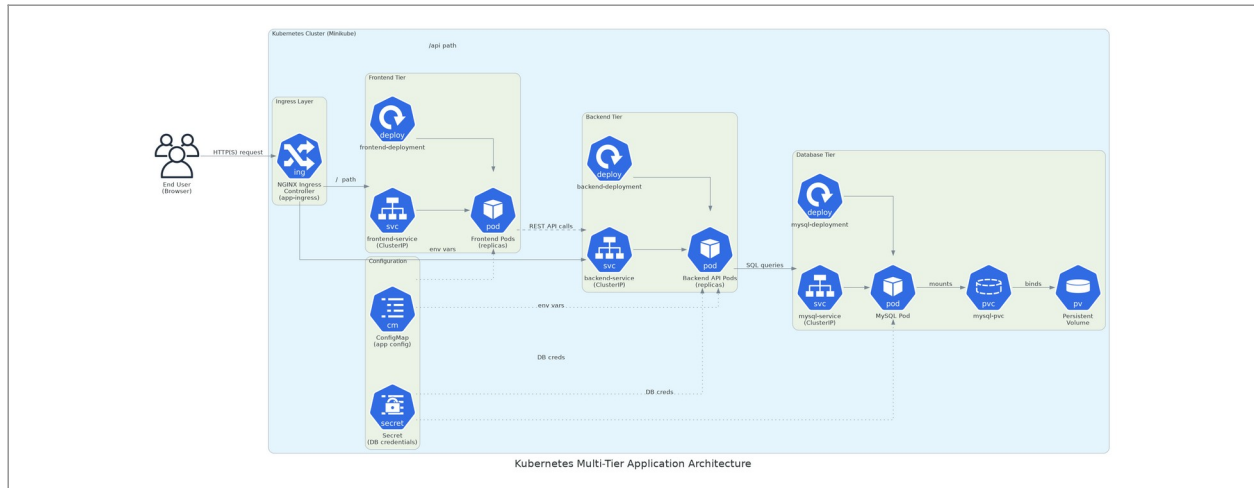


Figure 20: End-to-end system architecture of the multi-tier Kubernetes deployment

9. Deliverables

- Kubernetes YAML manifests (Deployments, Services, ConfigMaps, Secrets, PV/PVC, Ingress, Network Policies)
- Custom Helm Chart (final-project) with templated Deployments, Services, and Ingress
- Architecture diagram for the multi-tier application
- Deployment guide and README documentation
- Supporting screenshots of implementation and verification steps
- GitHub repository containing all project artifacts

10. Conclusion

Week 5 of the training program provided comprehensive, hands-on exposure to Kubernetes — progressing from fundamental cluster operations and Pod management through Services, configuration management, networking and security, observability, and Helm-based packaging, culminating in a fully functional multi-tier application deployment. This work strengthens the practical, production-oriented skill set required for a DevOps and Cloud Engineering role.